INtime 向け Rust と RTA ビルド

外部ライブラリ・アルゴリズム・データ構造・INtime.rs ラッパ・rtacc 手順

1. この文書の目的

`examples/rust/` 配下の INtime RTA 向け Rust サンプルを、次の観点で整理する。

- **Rust 外部クレート（ライブラリ）**を `no_std` / 32bit RTA で使うときの考え方
- **代表的なアルゴリズム**と、**FIFO・スタック・ヒープ**などリスト／コンテナのイメージ
- **`INtime.rs`** による INtime API のラッパ設計（FFI と RAII）
- **`rtacc`** を使った **`.rta` ビルド**の具体手順

PDF は同ディレクトリの `build_readme_pdf.ps1` で生成できる（Pandoc 等の有無に依存）。

---

2. ディレクトリ構成（抜粋）

パス	内容
`INtime.rs`	INtime `rt.lib` / `rt.h` 相当の **FFI と RAII ラッパ**（共有）
`main.rs` + `build_ll.ps1`	**メールボックス**サンプル（依存クレートなし、`rustc` 直叩きで `.ll`）
`rtacc_algorithms/`	**heapless / libm / micromath** を使うサンプル（**Cargo** で `.ll`）
`rtacc_algorithms/project.json`	`rtacc` 用プロジェクト定義
`../../bin/rtacc.exe`	リポジトリビルド済みのビルドドライバ（パスは環境による）

---

3. INtime 向け Rust の前提

- **ターゲット**: INtime RTA は **32bit（i386）** のため、`i686-pc-windows-msvc` でビルドする。
- **`#![no_std]`**: 標準ライブラリは使わない。ヒープは原則 **固定長** か INtime の `AllocateRtMemory` などに寄せる。
- **`#![no_main]`**: エントリは C ABI の `main`（`#[no_mangle] pub extern "C" fn main`）。

- **クレート種別**: staticlib として LLVM IR（.ll）を出し、rtacc が clang でオブジェクト化して clib / rt.lib とリンクして .rta を作る。
- **パニック**: 通常 **panic = "abort"**（Cargo.toml の [profile.*]）を指定する。
- **rustc の PATH**: Program Files 配下の単体 Rust には **i686 用 rustlib が無い**ことがある。**%USERPROFILE%¥.cargo¥bin を PATH 先頭**にし、**rustup 管理の** rustc を使うと安全。

---

4. 外部クレート（Cargo）の使い方

4.1 手順の概要

1. examples/rust/rtacc_algorithms/ で **cargo build --release**（.cargo/config.toml で i686-pc-windows-msvc を既定にできる）。
2. **cargo rustc --release -- --emit=llvm-ir** で依存クレートを同梱した **単一の .ll** を得る（build_ll.ps1 が deps から out/rust_rt_algorithms.ll にコピー）。
3. **rtacc project.json --toolset llvm** で out/app.rta を生成。

4.2 選定の目安

要件	例
no_std 対応	heapless, libm, micromath など **libm** は移植性の高い実装。
浮動小数	**micromath::F32Ext::sqrt** は **高速近似**（誤差あり）。検証は libm 側で行うのが無難
固定容量コンテナ	**heapless**（Vec, Deque, BinaryHeap, IndexMap 等）

4.3 main.rs だけのサンプル（Cargo なし）

rust_rt_mailbox は **rustc を直接**呼び、INtime.rs を #[path = "INtime.rs"] で取り込む。依存ゼロなので CI や最小再現に向く。

---

5. 代表的アルゴリズムとデータ構造（FIFO・スタック・ヒープ）

5.1 用語の整理

構造	特徴	典型操作
スタック（LIFO）	後入れ先出し	push / pop
キュー（FIFO）	先入れ先出し	enqueue / dequeue（push_back / pop_front 等）
二分ヒープ（優先度付きキュー）	優先度最大（または最小）を高速に取得	push / pop（根が最優先）
連結リスト	挿入・削除の位置を保ちやすい（std の LinkedList は no_std では非利用）	RTA では **固定配列＋インデックス** や **heapless** で代替することが多い

5.2 本リポジトリでの例

- **heapless::binary_heap::BinaryHeap<T, Max, N>**
  最大ヒープ。pop で大きい順に取り出す動きは、**ヒープソート**の「根を取り続ける」イメージに近い（rtacc_algorithms のデモ）。
- **FIFO / スタック**
  同じく **heapless::Deque** や **Vec** で両端操作すれば **キュー**にも**スタック**にもできる。サンプルコードは別途追加可能だが、**固定容量**と **オーバーフロー時の Result** に注意する。

5.3 数値まわり

- **libm::sqrtf**: no_std で一般的な **平方根**実装。
- **micromath**: 三角関数や **F32Ext::sqrt** など **組み込み向け近似**。sqrt はビット操作ベースで **誤差が大きい**ため、**厳密比較には使わない**。

---

6. INtime.rs ラッパの解説

INtime.rs は **no_std** のまま、INtime の RT API を **安全に近い形**で使うためのモジュールである。

6.1 FFI（extern "C"）

rt.h に相当する宣言を **mod ffi** に集約する。主なグループは次のとおり。

- **メモリ**: AllocateRtMemory, FreeRtMemory, CreateRtMemoryHandle, DeleteRtMemoryHandle, MapRtSharedMemory
- **メールボックス**: CreateRtMailbox, DeleteRtMailbox, SendRtHandle, ReceiveRtHandle, GetRtHandleType
- **スレッド・待機**: CreateRtThread, RtSleep
- **デバッグ出力**: printf

6.2 RtHandle

SDK の **RTHANDLE（u16）** をラップ。0xFFFF は無効として Option で表現する。

6.3 RtBuffer（AllocateRtMemory + 任意の CreateRtMemoryHandle）

- **allocate / allocate_zeroed:** バイトブロックを確保。ゼロ初期化が必要なら後者。
- **create_segment_handle:** 同一ブロックに対して **複数回**セグメントハンドルを作れる（上限 MAX_SEGMENT_HANDLES）。
- **Drop:** 登録した **すべての** DeleteRtMemoryHandle の後に **FreeRtMemory**（順序が SDK 要件と整合）。

6.4 RtMappedSharedView

MapRtSharedMemory の結果。**Drop で FreeRtMemory**（マッピング解除）。

6.5 RtMailbox

CreateRtMailbox。**Drop で DeleteRtMailbox**。

6.6 ヘルパ関数

- **zero_memory:** 生ポインタ範囲を fill(0)。
- **create_rt_thread, send_rt_handle, receive_rt_handle, is_memory_handle, rt_sleep**
- **printf_cstr, printf_summary:** フォーマット文字列は **ヌル終端**の &'static [u8]。

6.7 サンプルとの対応

サンプル	INtime.rs の主な利用
main.rs（メールボックス）	RtBuffer, RtMailbox, スレッド, Send/Receive, printf_summary 等
rtacc_algorithms	検証ロジックでは主に **printf_cstr**（他 API は未使用でもラッパを同梱可能）

---

7. RTA のビルド方法（rtacc）

7.1 LLVM（clang / lld）の導入

rtacc の **--toolset llvm** は **clang**（.ll → .asm / .obj）と **lld-link**（.rta リンク）を呼ぶ。Windows では次のいずれかで **LLVM の bin** を PATH に載せる（未設定時、rtacc は既定で C:¥Program Files¥LLVM¥bin を探す。環境変数 **LLVM** で上書き可）。

winget（推奨）

管理者不要のユーザースコープ例:

```
winget install -e --id LLVM.LLVM --scope user
```

システム全体に入れる場合:

```
winget install -e --id LLVM.LLVM
```

インストール後、**新しいターミナル**を開き、次で確認する。

```
where.exe clang
clang --version
```

`winget search LLVM` でバージョンや別パッケージを確認できる。パッケージ ID は環境により **`LLVM.LLVM`** が一般的（`winget show LLVM.LLVM` で説明を表示）。

その他の手法

方法	例
公式インストーラ	LLVM releases や llvm.org の Windows 用。既定パスは多くの場合 `C:¥Program Files¥LLVM¥bin`。
Chocolatey	管理者 PowerShell: `choco install llvm`
Scoop	`scoop install llvm`（ユーザーディレクトリ向け）
手動 PATH	LLVM の `bin` に `clang.exe` / `lld-link.exe` があることを確認し、**システムまたはユーザー PATH** に追加。

トラブルシュート（LLVM）

現象	対処
`clang` が見つからない	PATH を確認するか、**ユーザー環境変数** `LLVM` に `clang.exe` があるディレクトリを設定（例: `C:¥Program Files¥LLVM¥bin`）。
`rtacc` が別の clang を掴む	PATH の**先頭**に使いたい LLVM の `bin` を置く、または `LLVM` を明示。

7.2 事前準備（全体）

- **LLVM / clang**: 上記 **§7.1**。**clang** と **lld**（`lld-link`）が揃っていること。

- **Rust（rustup）**：サンプルは `rustup` の `rustc` + `i686-pc-windows-msvc` を前提とする。未導入なら rustup.rs か **winget**: `winget install -e --id Rustlang.Rustup` → 新しいターミナルで `rustup target add i686-pc-windows-msvc`。PATH は `%USERPROFILE%¥.cargo¥bin` を優先（`docs/GO_SETUP.md` も参照）。
- **INtime SDK**: リンカが `clib.lib`, `rt.lib` 等を参照できるように、通常は INtime のライブラリパスが通っていること（`rtacc` の出力ログに `LIBPATH` が出る）。
- **`bin/rtacc.exe`**: リポジトリルートで `build.ps1` または `go build -o bin/rtacc.exe ./cmd/rtacc` で生成。

7.3 メールボックス例（rustc → .ll）

```
cd examples¥rust
.¥build_ll.ps1
& ..¥..¥bin¥rtacc.exe project.json --toolset llvm
```

出力: `out/rust_rt_mailbox.ll` → 中間生成物 → **`out/app.rta`**（`project.json` の `sources` が参照する名前に合わせる）。

7.4 外部クレート例（Cargo → .ll）

```
cd examples¥rust¥rtacc_algorithms
.¥build_ll.ps1
& ..¥..¥..¥bin¥rtacc.exe project.json --toolset llvm
```

出力: **`out/rust_rt_algorithms.ll`, `out/app.rta`**。

7.5 project.json の要点

- **`sources`**: コンパイルする `.ll`（または `.c` 等）。
- **`output`**: 例として `app.rta`。
- **`toolset`**: `llvm` または `vs2017`。
- **`linker_flags`**: ヒープ／スタックサイズなど（サンプルに既定あり）。

詳細はリポジトリの **`docs/RTACC_USAGE.md`** を参照。

---

8. トラブルシュート（よくあること）

現象	対処のヒント
`can't find crate for core`（i686）	**rustup** で `i686-pc-windows-msvc` を入れ、**PATH で rustup の rustc を優先**
`micromath` の `sqrt` だけ値が大きくズレる	**近似実装**のため。厳密な検証は **`libm::sqrtf`** 等を使う

現象	対処のヒント
リンクで `___guard_check_icall_fptr` 重複警告	環境・SDK の組み合わせで出ることがある。`rtacc` が **Done** ならビルドは完了扱いのことが多い

---

9. 参考リンク

- TenAsys INtime SDK ドキュメント（各 API 名で検索）
- The Rust Embedded Book（`no_std` 全般）
- heapless, libm, micromath
- リポジトリ内: `docs/RTACC_USAGE.md`, `docs/GO_SETUP.md`

このファイルは `build_readme_pdf.ps1` から PDF または HTML に変換できる。